The fs Module: Reading & Writing Files

Why it matters

Almost every real backend touches the file system. You read a config file at boot, stream an uploaded avatar to disk, write structured logs, generate a PDF invoice, cache a rendered template, or serve a static asset. Node's built-in `fs` module is the single door to all of that, and how you open that door decides whether your server stays responsive under load or freezes for every user at once.

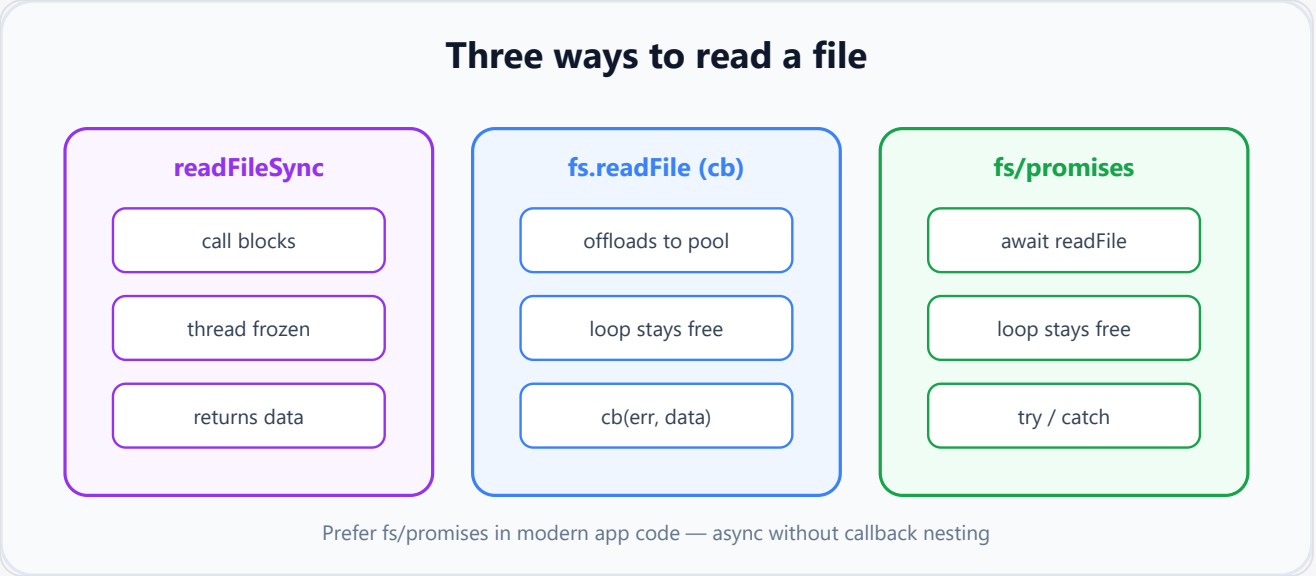
The `fs` module is also the place where the difference between *blocking* and *non-blocking* code becomes concrete and unforgiving. JavaScript on the server runs your application logic on a single main thread. A file read that blocks that thread does not just slow down the one request that asked for the file — it stops *every* concurrent request, every timer, and every incoming connection until the disk answers. Interviewers probe `fs` precisely because it is where candidates reveal whether they truly understand Node's execution model or have only memorised syntax. Getting this right is foundational; getting it wrong quietly caps your throughput at one slow disk operation at a time.

The concept, simply

The `fs` module ships in three flavours that do the same jobs with different control flow. **Synchronous** methods (`readFileSync`) run to completion and return the result directly, blocking the thread while they wait. **Callback** methods (`fs.readFile(path, cb)`) hand the work to Node's internal thread pool and call you back later with an error-first callback. **Promise** methods, imported from `node:fs/promises`, do the same non-blocking work but return a promise you can `await`, which reads like synchronous code but never blocks.

The mental model: sync is a phone call where you hold the line until you have your answer; async is texting a request and getting on with your day until a reply buzzes back.

On a server serving many people, you always want the second one.

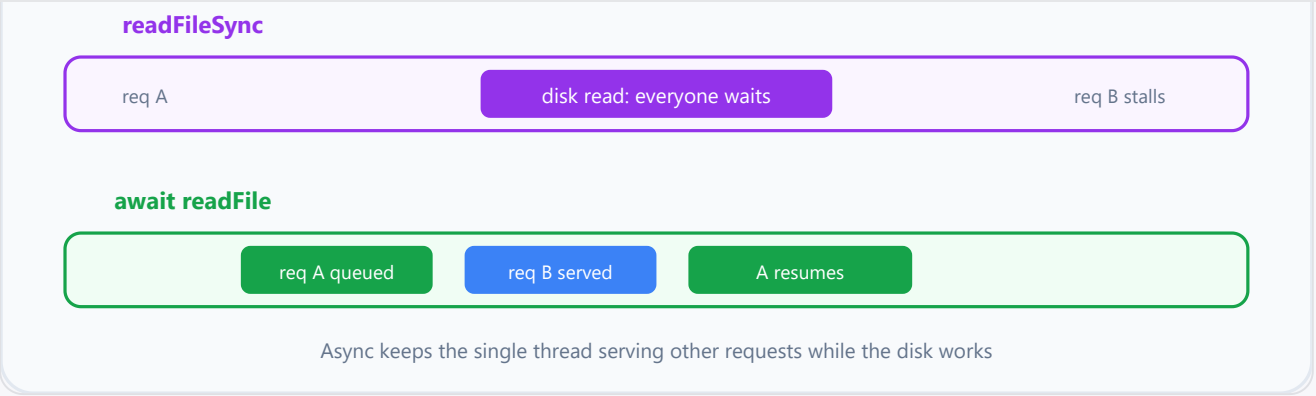


The same file read expressed three ways. Only the sync form freezes the thread; both async forms keep the event loop free.

How it works (deep)

When you call an asynchronous `fs` method, Node does not read the file on the main thread. File-system calls are handed to **libuv**, the C library underneath Node, which maintains a small pool of worker threads (four by default, tunable with the `UV_THREADPOOL_SIZE` environment variable). A worker performs the actual blocking `read()` or `write()` syscall while your main thread keeps running other JavaScript. When the worker finishes, libuv queues your callback (or resolves your promise) so the event loop can run it on the next available tick. This is why "non-blocking" is accurate from JavaScript's point of view even though the underlying OS call is blocking — the blocking simply happens on a thread that is not yours.

Synchronous methods skip all of that. `readFileSync` performs the syscall directly on the main thread and does not return until the kernel hands back the bytes. During that window nothing else in your process advances: no HTTP requests are accepted, no timers fire, no promises resolve. On a laptop reading a 2 KB config this is microseconds and invisible; under a server handling hundreds of requests per second against a slow network drive it is a self-inflicted outage.



Sync I/O serialises everyone behind the disk. Async lets the single thread serve other requests while the read is in flight.

The `fs/promises` API is not a thin wrapper you bolt on yourself — it is a first-class, separately maintained surface (`node:fs/promises`) that returns native promises directly from libuv completions. Under the hood it uses the same thread pool as the callback API, so it carries no extra cost; it simply gives you `async/await` ergonomics, real `try/catch` error handling, and clean composition with other async work. For code written today, it is the default you should reach for.

One more layer worth knowing: reading an entire file with `readFile` buffers the whole thing into memory before you see a single byte. That is perfect for small files but ruinous for large ones — a 2 GB file becomes a 2 GB allocation. For large or unbounded data you switch to **streams** (`fs.createReadStream` / `createWriteStream`), which move data in small chunks with built-in back-pressure so memory stays flat regardless of file size.

Code examples

Reading and writing with the modern promise API, with proper error handling:

JAVASCRIPT Copy

```
import { readFile, writeFile } from 'node:fs/promises';

async function loadConfig() {
  try {
    const raw = await readFile('config.json', 'utf8'); // string, not Buff
    return JSON.parse(raw);
  } catch (err) {
```

```
    }
    throw err; // Anything else (permissions, corrupt JSON) is a real erro
  }
}

async function saveConfig(cfg) {
  await writeFile('config.json', JSON.stringify(cfg, null, 2), 'utf8');
}
```

Passing the `'utf8'` encoding makes `readFile` return a decoded string. Omit it and you get a raw `Buffer` instead — useful for binary data, wrong for JSON parsing.

Appending log lines, creating directories, and checking existence the right way:

JAVASCRIPT Copy

```
import { appendFile, mkdir, access, constants } from 'node:fs/promises';

// mkdir with recursive:true is idempotent — safe to call every boot.
await mkdir('logs', { recursive: true });

// Append never truncates; it creates the file if absent.
await appendFile('logs/app.log', `${new Date().toISOString()} started
`);

// Prefer trying the operation over checking first (see gotchas: TOCTOU).
async function canRead(path) {
  try {
    await access(path, constants.R_OK);
    return true;
  } catch {
    return false;
  }
}
```

Streaming a large file instead of buffering it, using the pipeline helper that wires up back-pressure and error propagation for you:

```
import { createReadStream, createWriteStream } from 'node:fs';
import { pipeline } from 'node:stream/promises';
import { createGzip } from 'node:zlib';

// Compress a 5 GB file with flat memory — chunks flow through, nothing bu
await pipeline(
  createReadStream('backup.tar'),
  createGzip(),
  createWriteStream('backup.tar.gz')
);
console.log('done — memory stayed constant regardless of file size');
```

Watching a file for changes, for a dev-time reload or a config hot-reload:

JAVASCRIPT Copy

```
import { watch } from 'node:fs/promises';

// Async-iterator watcher: clean, cancellable with an AbortSignal.
const ac = new AbortController();
setTimeout(() => ac.abort(), 60_000); // stop watching after a minute

try {
  for await (const event of watch('config.json', { signal: ac.signal })) {
    console.log('config changed:', event.eventType);
  }
} catch (err) {
  if (err.name !== 'AbortError') throw err;
}
```

Common mistakes & gotchas

- **Using `*Sync` methods inside a request handler.** It works in every test with one user and collapses under real concurrency because it blocks the whole event loop. Sync is fine at startup or in CLI scripts; it is a bug in a hot server path.
- **Forgetting the encoding and being surprised by a `Buffer`.** `await readFile('x.txt')` returns a `Buffer`; `JSON.parse` on it happens to work (it

- **The check-then-act (TOCTOU) race.** Calling `existsSync` or `access` and *then* reading/writing invites a race: the file can vanish or appear between the two calls. Just attempt the operation and handle the error code (`ENOENT`, `EEXIST`). `fs.exists` is deprecated for exactly this reason.
- **Swallowing the error-first callback's first argument.** In callback style, `(err, data) => { use(data) }` that ignores `err` will read `undefined` data and crash later with a confusing stack. Always check `err` first.
- **Reading a huge file with `readFile`.** It allocates the entire file in memory at once. A few concurrent large reads can OOM the process. Stream anything that is not comfortably small and bounded.
- **Assuming `writeFile` is atomic.** A crash mid-write can leave a truncated file. For durability, write to a temp file and `rename` it into place — `rename` is atomic on the same filesystem.
- **Relative paths depending on the current working directory.** `readFile('data.json')` resolves against `process.cwd()`, not the file's own folder, so it breaks when the process is launched from elsewhere. Anchor paths to `import.meta.dirname` (or `__dirname`).

Best practices

- **Default to `node:fs/promises` with `async/await`.** It is non-blocking, composes cleanly, and gives you real `try/catch`.
- **Reserve `*Sync` for startup and scripts.** Loading config before the server listens, or one-off CLI tooling, is exactly where sync is acceptable and even clearer.
- **Stream anything large or of unknown size,** using `stream/promises` `pipeline` so errors and back-pressure are handled for you.
- **Branch on `err.code`, not on message strings.** Codes like `ENOENT`, `EACCES`, `EEXIST` are stable; human-readable messages are not.
- **Build paths with the `path` module and anchor to `import.meta.dirname`,** never by concatenating strings with `/`.

- **Prefer attempting an operation over pre-checking existence,** to avoid TOCTOU races and an extra syscall.

⭐ Interview lens

Q: What is the real difference between `fs.readFileSync` and `fs.readFile`? `readFileSync` blocks the main thread until the OS returns the bytes, so nothing else in the process runs meanwhile. `readFile` hands the work to libuv's thread pool and invokes your callback later, keeping the event loop free to serve other requests. The trap is claiming sync is "faster" — for one isolated read the latency is similar, but on a server sync destroys concurrency.

Q: If Node is single-threaded, how does async file I/O actually run in parallel? Your JavaScript is single-threaded, but I/O is delegated to libuv's thread pool (default size 4). Those worker threads perform the blocking syscalls; when they finish, the completion callback is queued back onto the single main thread. So the parallelism is in the C layer, not in your JS.

Q: Why prefer `fs/promises` over the callback API if they use the same thread pool? They have identical performance because the mechanism is the same. The win is purely ergonomic: `await` avoids callback nesting, `try/catch` handles errors naturally, and it composes with other promises. There is no runtime downside.

Q: You must read a 4 GB file and transform it. How? Never `readFile` it — that would try to allocate 4 GB. Use `createReadStream` and process chunks, ideally via `pipeline` so back-pressure and error handling are automatic. Memory stays flat regardless of file size.

you should not. Pre-checking with `access` / `existsSync` then reading opens a TOCTOU race and costs an extra syscall. Just `readFile` and catch `err.code === 'ENOENT'`. This shows you understand races, not just APIs.

Q: How would you make a config write crash-safe? Write to a temporary file in the same directory, then `rename` it over the target. `rename` within one filesystem is atomic, so a reader ever sees either the old file or the fully new one, never a truncated write.

Cheat-card

- `import { readFile, writeFile } from 'node:fs/promises'` — the default modern API.
- Pass `'utf8'` for a string; omit it for a raw `Buffer`.
- Async = libuv thread pool (default 4, set `UV_THREADPOOL_SIZE`); main thread stays free.
- `*Sync` only at startup / in scripts — never in a request handler.
- Large or unbounded data → `createReadStream` + `pipeline`, not `readFile`.
- Switch on `err.code`: `ENOENT` (missing), `EACCES` (permissions), `EEXIST` (already there).
- `mkdir(dir, { recursive: true })` is idempotent; `appendFile` creates + never truncates.
- Crash-safe write = temp file + atomic `rename`.
- Anchor paths to `import.meta.dirname`, never `process.cwd()`.

QUICK CHECK — no marks, just for you.

1. Why is calling fs.readFileSync inside an HTTP request handler considered a bug on a busy server?

It blocks the single main thread, so no other request, timer, or connection advances until the disk returns

- It always throws because synchronous methods are forbidden outside of startup code
- It reads the file more slowly than the async version for that one read
- It corrupts the file because synchronous writes are not atomic

Check answer

2. If Node runs your JavaScript on a single thread, how does asynchronous fs I/O run without blocking it?

- V8 spawns a new JavaScript thread for each file operation
- libuv hands the blocking syscall to a thread pool (default 4), then queues your callback back onto the main thread when it finishes
- The operating system runs the callback directly on a kernel thread
- The fs/promises API uses a separate faster syscall that never blocks

Check answer

3. What is the recommended way to check whether a file exists before reading it, per the lesson?

- Call fs.existsSync first, then readFile only if it returns true
- Call access with R_OK, and if it resolves you are guaranteed the read will succeed
- Usually you should not pre-check; just attempt the read and handle err.code === "ENOENT"
- Use fs.exists, which is the modern non-deprecated existence check

Previous
EventEmitter

Next
path, os & URL: Portable Filesystem and Environment Info

LEARN MORE

ARTICLES & DOCS

- File system | Node.js Documentation
  nodejs.org
- Reading files with Node.js
  nodejs.org
- Working with file descriptors in Node.js
  nodejs.org

VIDEOS



A Beginner's Guide To The File System Module In Node.js
Catalin Pit

Hand-picked references for this lesson.